

MATRIXFLOW

ENTERPRISE

Sistema Web Empresarial de Análisis de Ventas,
Inventario e Indicadores mediante Álgebra Lineal

PLAN MAESTRO DE DESARROLLO

Frontend React + TypeScript | Backend Python + FastAPI | PostgreSQL | NumPy

Documento técnico empresarial
Versión 1.0 • Septiembre 2026

CONTROL DOCUMENTAL

Elemento	Definición
Documento	Plan Maestro de Desarrollo – MatrixFlow Enterprise
Versión	1.0
Fecha	Septiembre 2026
Tipo	Documento técnico y de planificación
Frontend	React + TypeScript + Vite
Backend	Python + FastAPI
Base de datos	PostgreSQL
Motor matemático	Python + NumPy
Metodología	Ágil / Scrum
Tema académico	Álgebra lineal, vectores y matrices

Base documental: el proyecto original define MatrixFlow Enterprise como una aplicación web empresarial para registrar, administrar y analizar vectores y matrices, con React + TypeScript, FastAPI + Python, NumPy y PostgreSQL. El presente documento reorganiza y concreta esa propuesta para un escenario empresarial de ventas e inventario, manteniendo el contenido matemático como núcleo del sistema. [filecite?turn0file0?L30-L42?](#)

ÍNDICE DEL DOCUMENTO

- 1. Resumen ejecutivo
- 2. Concepto y alcance del sistema
- 3. Problema y justificación
- 4. Objetivos
- 5. Modelo de negocio y escenario empresarial
- 6. Arquitectura general
- 7. Fases de desarrollo
- 8. Fase 1 – Frontend React + TypeScript
- 9. Fase 2 – Backend Python + FastAPI
- 10. Fase 3 – PostgreSQL y modelo de datos
- 11. Fase 4 – Motor matemático Python + NumPy
- 12. Fase 5 – Integración
- 13. Fase 6 – Seguridad y auditoría
- 14. Fase 7 – Dashboard y reportes
- 15. Fase 8 – Pruebas
- 16. Metodología Scrum y sprints
- 17. Estructura técnica del proyecto
- 18. Requerimientos funcionales y no funcionales
- 19. Criterios de aceptación
- 20. Entregables y roadmap
- 21. Presentación académica sugerida

1. RESUMEN EJECUTIVO

MatrixFlow Enterprise será un sistema web empresarial orientado al análisis de ventas, inventario e indicadores de una empresa con múltiples sucursales. Su característica diferencial es que la información empresarial se transformará en vectores y matrices para ejecutar operaciones de álgebra lineal y convertir sus resultados en indicadores y reportes.

La propuesta mantiene el enfoque académico de la Semana 06: operaciones con vectores, matrices, suma, resta, multiplicación, transposición, producto escalar, multiplicación por escalar y combinaciones lineales. El documento fuente establece que el sistema debe hacer visible el uso de estos contenidos y que Python debe realizar los cálculos. [filecite?turn0file0L427-L435](#)

2. CONCEPTO Y ALCANCE DEL SISTEMA

El sistema no será una calculadora matricial aislada. Será una plataforma empresarial donde las operaciones matemáticas forman parte de los procesos de análisis.

Área	Función
Empresa	Información corporativa y configuración del negocio.
Sucursales	Administración de sedes y datos operativos.
Productos	Catálogo, categorías y variables de análisis.
Ventas	Registro y análisis de cantidades e importes.
Inventario	Existencias y movimientos.
Vectores	Representación de información empresarial unidimensional.
Matrices	Representación de información multidimensional.
Operaciones	Cálculos de álgebra lineal.
Historial	Trazabilidad de cálculos y resultados.
Reportes	Indicadores, gráficos y exportaciones.

3. PROBLEMA Y JUSTIFICACIÓN

Una empresa con varias sucursales necesita analizar simultáneamente ventas, productos, costos, inventario y metas. El crecimiento de los datos dificulta realizar comparaciones y cálculos manuales. MatrixFlow centralizará la información y proporcionará un mecanismo matemático reproducible para transformar datos empresariales en resultados analíticos.

El documento base propone como caso de uso que una empresa registre indicadores por sucursal o producto en matrices, ejecute operaciones y conserve el resultado de manera reproducible. [filecite?turn0file0L37-L42](#)

4. OBJETIVOS

4.1 Objetivo general

Desarrollar un sistema web empresarial que permita registrar, organizar y analizar información de ventas, productos, inventario y sucursales mediante vectores y matrices, aplicando operaciones de álgebra lineal con Python y NumPy para generar indicadores, análisis y reportes.

4.2 Objetivos específicos

- Registrar empresas, sucursales, productos, usuarios, ventas e inventario.
- Representar información empresarial mediante vectores.
- Representar información multidimensional mediante matrices.
- Aplicar suma, resta y multiplicación por escalar.
- Aplicar producto escalar y multiplicación matricial.
- Aplicar transposición y combinaciones lineales.
- Validar dimensiones y datos antes de ejecutar operaciones.
- Guardar entradas, resultados, usuario, fecha y estado de cada operación.
- Generar indicadores y reportes empresariales.
- Implementar seguridad, roles y auditoría.

5. MODELO DE NEGOCIO Y ESCENARIO EMPRESARIAL

Escenario de referencia: empresa comercial con múltiples sucursales y productos tecnológicos. El modelo permitirá registrar ventas por sucursal y producto, inventario y metas. Esta información se utilizará como fuente para la construcción de vectores y matrices.

Entidad	Ejemplo
Sucursal	Lima, Arequipa, Trujillo, Cusco, Piura
Producto	Laptop, PC, Monitor, Teclado, Mouse
Vector	Ventas de una sucursal por producto
Matriz	Ventas de todas las sucursales por producto
Producto escalar	Cantidades $\times$ precios = ingresos
Resta matricial	Ventas reales – metas
Escalar	Ajuste porcentual de precios o cantidades
Combinación lineal	Indicador ponderado de ventas, rentabilidad y rotación

6. ARQUITECTURA GENERAL



La arquitectura conserva la separación de responsabilidades del documento fuente: presentación, servicios frontend, API, servicios backend, algoritmos y persistencia. [filecite?turn0file0?L63-L70?](#)

7. FASES DE DESARROLLO

Fase	Objetivo	Tecnología principal	Entregable
1	Frontend empresarial	React + TypeScript + Vite	UI, navegación y módulos
2	Backend API	Python + FastAPI	API REST y servicios
3	Persistencia	PostgreSQL + SQLAlchemy	Modelo de datos y migraciones
4	Álgebra lineal	Python + NumPy	Algoritmos matemáticos
5	Integración	React $\leftrightarrow$ FastAPI	Sistema conectado
6	Seguridad	JWT + RBAC	Autenticación y permisos
7	Analítica	Recharts + API	Dashboard y reportes
8	Calidad	Pytest + pruebas UI/API	Sistema validado

8. FASE 1 – FRONTEND REACT + TYPESCRIPT

Esta será la primera fase de implementación. El objetivo es construir una experiencia empresarial completa, con navegación, layout, formularios, tablas, dashboard y pantallas de análisis. El documento fuente establece iniciar por el frontend y utilizar datos simulados antes de conectar FastAPI. [filecite?turnOfile0?L149-L151?](#)

8.1 Tecnologías

Tecnología	Uso
React + TypeScript	Interfaz y tipado.
Vite	Desarrollo y build.
TanStack Query	Consultas y caché del servidor.
Axios	Comunicación HTTP.
Tailwind CSS / shadcn/ui	Sistema visual.
Lucide React	Iconografía.
React Hook Form + Zod	Formularios y validación.
Recharts	Visualización de indicadores.

8.2 Navegación

MATRIXFLOW
├─ Dashboard
├─ Empresa
│ └─ Sucursales
│ └─ Productos
├─ Ventas
├─ Inventario
├─ Análisis Matemático
│ └─ Vectores
│ └─ Matrices
│ └─ Operaciones
│ └─ Combinaciones lineales
├─ Historial
├─ Reportes
├─ Usuarios
└─ Configuración

8.3 Pantallas de la Fase 1

- /login – autenticación.
- /dashboard – indicadores ejecutivos.
- /empresa – información de la empresa.
- /sucursales – gestión de sucursales.
- /productos – catálogo.
- /ventas – registro y consulta.
- /inventario – existencias y movimientos.
- /vectores – gestión visual de vectores.
- /matrices – cuadrícula editable de matrices.
- /operaciones – selección y ejecución de cálculos.
- /historial – trazabilidad.
- /reportes – indicadores y gráficos.
- /configuracion – parámetros.

8.4 Identidad visual

Elemento	Propuesta
Fondo	#F8FAFC
Sidebar	#0F172A
Primario	#2563EB
Acento	#06B6D4
Texto	#0F172A
Secundario	#64748B
Estilo	Enterprise / SaaS / Data Analytics

8.5 Entregables

- Layout empresarial responsive.
- Sidebar y navegación.
- Login visual.
- Dashboard con datos simulados.
- CRUD visual de empresa, sucursales y productos.
- Pantallas de ventas e inventario.
- Editor visual de vectores y matrices.
- Pantalla de operaciones.
- Historial y reportes.

9. FASE 2 – BACKEND PYTHON + FASTAPI

FastAPI será la capa de servicios del sistema. Pydantic validará los datos, SQLAlchemy gestionará la persistencia y los algoritmos estarán separados de los routers. Esta separación coincide con la arquitectura propuesta en el documento fuente.

[filecite?url=urn:file:000L240-L257](#)

9.1 Estructura

```
backend/  
└─ app/  
    └─ api/routes/  
        └─ auth.py  
        └─ users.py  
        └─ companies.py  
        └─ branches.py  
        └─ products.py  
        └─ sales.py  
        └─ inventory.py  
        └─ vectors.py  
        └─ matrices.py  
        └─ operations.py  
        └─ reports.py  
    └─ core/  
    └─ models/  
    └─ schemas/  
    └─ services/  
    └─ repositories/  
    └─ algorithms/  
    └─ main.py
```

9.2 API inicial

Endpoint	Propósito
POST /api/v1/auth/login	Autenticación
GET /api/v1/companies	Empresas
GET /api/v1/branches	Sucursales
GET /api/v1/products	Productos
GET /api/v1/sales	Ventas
GET /api/v1/inventory	Inventario
POST /api/v1/vectors	Crear vector
POST /api/v1/matrices	Crear matriz
POST /api/v1/operations	Ejecutar operación
GET /api/v1/operations	Historial
GET /api/v1/reports	Reportes

9.3 Flujo de una operación



El documento fuente presenta este mismo patrón de validación, servicio, NumPy, historial y respuesta JSON.
[filecite\turn0file0\277-L315](#)

10. FASE 3 – POSTGRESQL Y MODELO DE DATOS

PostgreSQL será la base de datos principal. La persistencia debe representar los procesos empresariales y también conservar las entradas y resultados de las operaciones matemáticas.

Tabla	Responsabilidad
users	Usuarios del sistema.
roles	Roles y permisos.
companies	Empresas.
branches	Sucursales.
categories	Categorías de productos.
products	Catálogo.
sales	Cabecera de ventas.
sale_details	Detalle de productos vendidos.
inventory	Existencias.
inventory_movements	Movimientos.
targets	Metas empresariales.
vectors	Vectores.
vector_values	Valores de vectores.
matrices	Matrices.
matrix_values	Valores matriciales.
operations	Operaciones ejecutadas.
operation_inputs	Entradas de operaciones.
operation_results	Resultados.
audit_logs	Auditoría.

11. FASE 4 – MOTOR MATEMÁTICO PYTHON + NUMPY

Esta fase materializa directamente la Semana 06. Los algoritmos deben ser independientes, comprobables y reutilizables. El documento fuente define módulos para vectores, matrices y álgebra lineal. [filecite\turn0file0\277-L282](#)

11.1 Vectores

```
sum_vector()
subtract_vector()
scalar_multiply()
dot_product()
```

11.2 Matrices

```
add_matrix()
subtract_matrix()
multiply_matrix()
transpose_matrix()
scalar_multiply_matrix()
```

11.3 Álgebra lineal

```
linear_combination()
validate_dimensions()
validate_vector()
validate_matrix()
```

11.4 Aplicaciones empresariales

Operación	Aplicación
Suma	Acumulado de ventas de períodos.
Resta	Ventas reales frente a metas.
Escalar	Ajustes porcentuales.
Producto escalar	Cantidades × precios.
Multiplicación	Transformación de cantidades a indicadores.
Transpuesta	Cambio de perspectiva sucursal/producto.
Combinación lineal	Indicador empresarial ponderado.

12. FASE 5 – INTEGRACIÓN

La integración conectará el frontend, API, motor matemático y PostgreSQL. El frontend no accederá directamente a la base de datos; utilizará la API.

```
React
↓
TanStack Query / Axios
↓
FastAPI
↓
Services
├─ Business Logic
├─ NumPy
↓
PostgreSQL
↓
JSON
↓
React
```

13. FASE 6 – SEGURIDAD Y AUDITORÍA

Rol	Acceso
Administrador	Usuarios, empresa, ventas, inventario, matrices, vectores, operaciones, reportes y configuración.
Analista	Ventas, inventario, matrices, vectores, operaciones y reportes.
Consulta	Dashboard y reportes autorizados.

El documento base contempla administrador, analista y consulta, además del registro de usuario, acción, módulo, fecha, IP cuando corresponda, estado y resultado. [filecite?turn0file0?L318-L332](#)

14. FASE 7 – DASHBOARD Y REPORTES

- Ventas por sucursal.
- Ventas por producto.
- Cumplimiento de metas.

- Inventario y rotación.
- Resultados de operaciones matemáticas.
- Actividad reciente.
- Indicadores de procesamiento.
- Exportaciones futuras.

15. FASE 8 – PRUEBAS

Tipo	Objetivo
Unitarias	Validar cada algoritmo matemático.
Integración	Validar React ↔ API ↔ PostgreSQL.
API	Validar endpoints, esquemas y errores.
UI	Validar formularios, navegación y estados.
Seguridad	Validar autenticación y permisos.
Aceptación	Comprobar requisitos del sistema.

Estas categorías de pruebas están contempladas en el documento original. [filecite\[turn0file0L407-L414\]](#)

16. METODOLOGÍA SCRUM Y SPRINTS

Sprint	Objetivo	Entregable principal
1	Diseño y frontend base	Layout, login, navegación y dashboard.
2	Módulos empresariales	Empresa, sucursales, productos, ventas e inventario.
3	Frontend matemático	Vectores, matrices, operaciones e historial.
4	Backend	FastAPI, endpoints y servicios.
5	Base de datos	PostgreSQL, modelos y migraciones.
6	Motor matemático	Python + NumPy y pruebas unitarias.
7	Integración	React + FastAPI + PostgreSQL + NumPy.
8	Seguridad y auditoría	JWT, roles, permisos y trazabilidad.
9	Reportes	Dashboard, gráficos y exportaciones.
10	Pruebas y documentación	Validación final y entrega.

La planificación conserva el enfoque Scrum del documento fuente, que propone entregables funcionales por sprint y validación progresiva. [filecite\[turn0file0L335-L358\]](#)

17. ESTRUCTURA TÉCNICA DEL PROYECTO

```
matrixflow-enterprise/
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/
│   │   ├── types/
│   │   ├── schemas/
│   │   ├── main.tsx
│   │   └── package.json
│   └── backend/
│       ├── app/
│       │   ├── api/
│       │   ├── core/
│       │   ├── models/
│       │   ├── schemas/
│       │   ├── services/
│       │   ├── repositories/
│       │   ├── algorithms/
│       │   └── main.py
│       ├── requirements.txt
│       └── alembic.ini
├── database/
├── docs/
├── .env.example
├── docker-compose.yml
└── README.md
```

18. REQUERIMIENTOS FUNCIONALES

- RF-01: Iniciar sesión.
- RF-02: Gestionar usuarios y roles.
- RF-03: Gestionar empresas y sucursales.
- RF-04: Gestionar productos y categorías.
- RF-05: Registrar ventas.
- RF-06: Gestionar inventario.
- RF-07: Registrar metas.
- RF-08: Crear y consultar vectores.
- RF-09: Crear y consultar matrices.
- RF-10: Ejecutar operaciones vectoriales.
- RF-11: Ejecutar operaciones matriciales.
- RF-12: Ejecutar combinaciones lineales.
- RF-13: Conservar historial.
- RF-14: Generar reportes.
- RF-15: Registrar eventos de auditoría.

19. REQUERIMIENTOS NO FUNCIONALES

- Seguridad y control de acceso.
- Arquitectura modular.

- Separación frontend/backend.
- Validación en frontend y backend.
- Persistencia estructurada.
- Trazabilidad de operaciones.
- Escalabilidad y mantenibilidad.
- Interfaz responsive.
- Rendimiento adecuado para operaciones matriciales.

20. CRITERIOS DE ACEPTACIÓN

Código	Criterio
CA-01	El usuario puede autenticarse según su rol.
CA-02	El administrador puede registrar sucursales y productos.
CA-03	El usuario autorizado puede registrar ventas.
CA-04	El sistema permite representar datos como vectores.
CA-05	El sistema permite representar datos como matrices.
CA-06	Las dimensiones incompatibles son rechazadas.
CA-07	Las operaciones producen resultados matemáticamente válidos.
CA-08	Cada operación queda almacenada en el historial.
CA-09	El resultado puede visualizarse en el frontend.
CA-10	Los reportes utilizan datos persistidos.

21. ENTREGABLES Y ROADMAP

Entregable	Resultado
E01	Documento de requisitos.
E02	Diseño UI/UX empresarial.
E03	Frontend React + TypeScript.
E04	Backend Python + FastAPI.
E05	Base PostgreSQL.
E06	Motor de álgebra lineal con NumPy.
E07	Integración completa.
E08	Seguridad y auditoría.
E09	Dashboard y reportes.
E10	Pruebas y documentación final.

22. PRESENTACIÓN ACADÉMICA SUGERIDA

Sección	Contenido
1. Planteamiento	Problema empresarial.
2. Justificación	Importancia de centralizar y analizar información.
3. Objetivos	Objetivo general y específicos.
4. Requerimientos	Funcionales y no funcionales.
5. Arquitectura	React, FastAPI, NumPy y PostgreSQL.
6. Diseño	UI/UX, módulos y navegación.
7. Desarrollo	Frontend, backend y base de datos.
8. Matemática	Vectores, matrices y operaciones.
9. Aplicación empresarial	Ventas, inventario e indicadores.
10. Pruebas	Casos y resultados.
11. Conclusiones	Resultados y trabajo futuro.

La estructura mantiene la presentación académica sugerida en el documento fuente, incluyendo planteamiento, justificación, objetivos, requerimientos, arquitectura, UI/UX, desarrollo, matemática, pruebas y conclusiones. [filecite?turnOfile0?L415-L426](#)

23. CONCLUSIÓN DEL PLAN

MatrixFlow Enterprise queda definido como una solución empresarial en la que el álgebra lineal no es un módulo independiente sin contexto, sino el mecanismo matemático que permite transformar datos de ventas, productos, inventario y metas en resultados analíticos.

El desarrollo comenzará por el frontend React + TypeScript, continuará con el backend Python + FastAPI y la persistencia PostgreSQL, y posteriormente integrará el motor matemático Python + NumPy. Esta secuencia conserva la estrategia por fases del documento original y permite validar cada componente antes de la integración completa. [filecite?turnOff=0&L438-L446](#)